

SHA256 알고리즘 & 양자회로 분석

<https://youtu.be/3cr9l8oT9T0>

정보컴퓨터공학과 송경주

SHA256 Structure

- 초기화 (SHA256_Init())
 - 초기 해시값 설정
 - $H[0] = 0x6a09e667$, $H[1] = 0xbb67ae85$, $H[2] = 0x3c6ef372$, $H[3] = 0xa54ff53a$,
 - $H[4] = 0x510e527f$, $H[5] = 0x9b05688c$, $H[6] = 0x1f83d9ab$, $H[7] = 0x5be0cd19$
- 업데이트 함수(SHA256_Update())
 - 64 바이트 블록 단위로 자름
 - 압축함수 호출
- 압축함수 (Compression)
 - 64 Round function
- Final function

```
/* Ch (Choose): x가 1이면 y 선택, 0이면 z 선택 */
#define Ch(x, y, z) (((x) & (y)) ^ ((~(x)) & (z)))

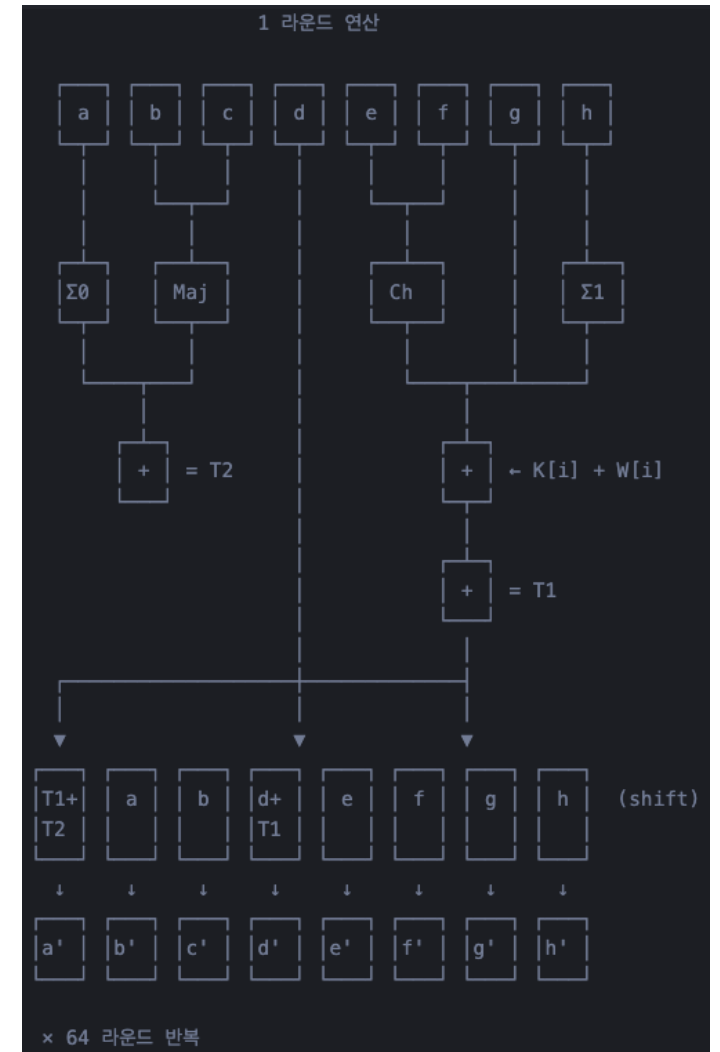
/* Maj (Majority): x, y, z 중 다수결 */
#define Maj(x, y, z) (((x) & (y)) ^ ((x) & (z)) ^ ((y) & (z)))

/* Σ0 (대문자 시그마 0): 압축 함수용 */
#define Sigma0(x) (ROTR(x, 2) ^ ROTR(x, 13) ^ ROTR(x, 22))

/* Σ1 (대문자 시그마 1): 압축 함수용 */
#define Sigma1(x) (ROTR(x, 6) ^ ROTR(x, 11) ^ ROTR(x, 25))

/* σ0 (소문자 시그마 0): 메시지 스케줄용 */
#define sigma0(x) (ROTR(x, 7) ^ ROTR(x, 18) ^ SHR(x, 3))

/* σ1 (소문자 시그마 1): 메시지 스케줄용 */
#define sigma1(x) (ROTR(x, 17) ^ ROTR(x, 19) ^ SHR(x, 10))
```



Compression function in SHA256

$T1 = h + \Sigma_1(e) + Ch(e,f,g) + K[i] + W[i]$

$T2 = \Sigma_0(a) + Maj(a,b,c)$

$h = g$

$g = f$

$f = e$

$e = d + T1$

$d = c$

$c = b$

$b = a$

$a = T1 + T2$

```

/* Ch (Choose): x가 1이면 y 선택, 0이면 z 선택 */
#define Ch(x, y, z)  (((x) & (y)) ^ ((~(x)) & (z)))

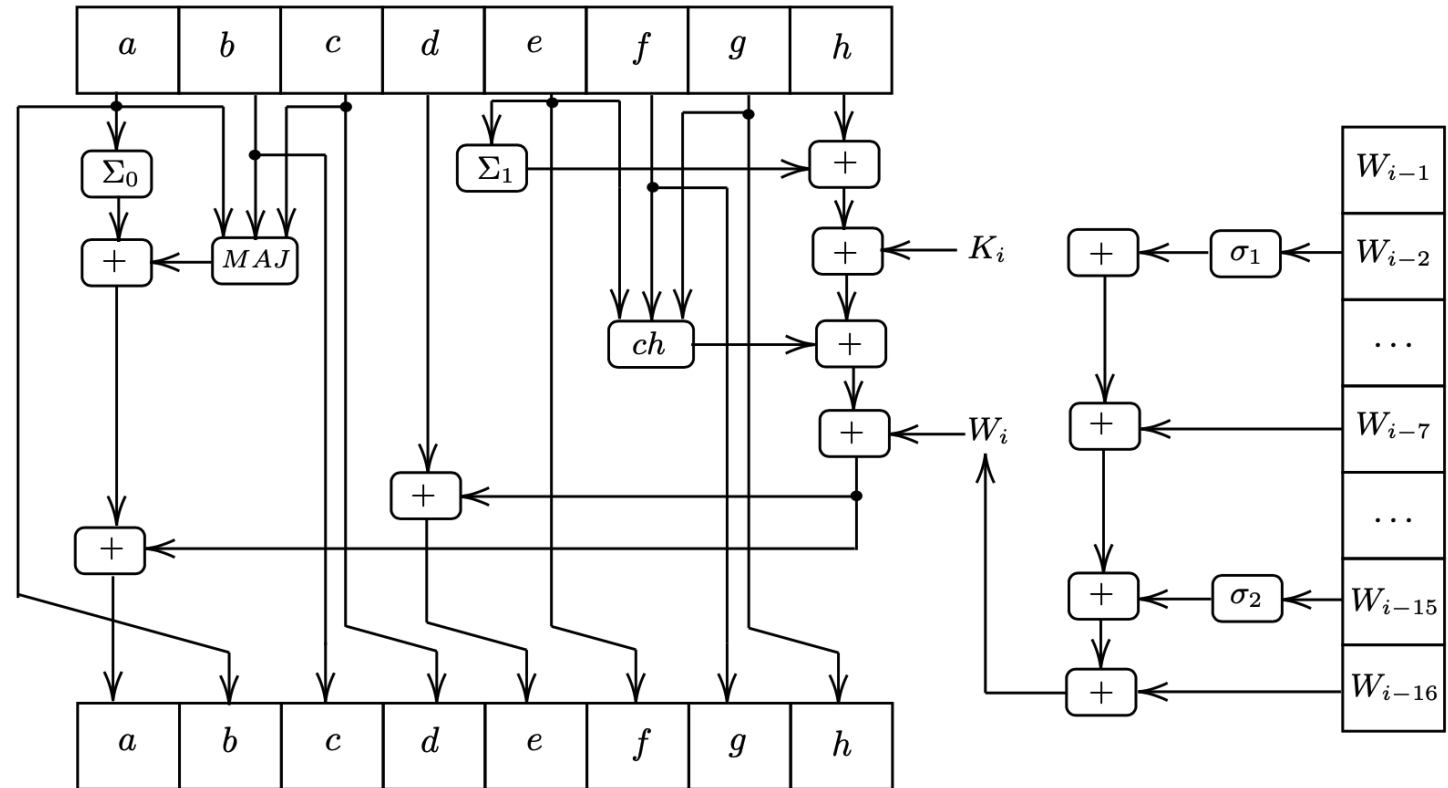
/* Maj (Majority): x, y, z 중 다수결 */
#define Maj(x, y, z) (((x) & (y)) ^ ((x) & (z)) ^ ((y) & (z)))

/* Σ0 (대문자 시그마 0): 압축 함수용 */
#define Sigma0(x)    (ROTR(x, 2) ^ ROTR(x, 13) ^ ROTR(x, 22))

/* Σ1 (대문자 시그마 1): 압축 함수용 */
#define Sigma1(x)    (ROTR(x, 6) ^ ROTR(x, 11) ^ ROTR(x, 25))

/* σ0 (소문자 시그마 0): 메시지 스케줄용 */
#define sigma0(x)    (ROTR(x, 7) ^ ROTR(x, 18) ^ SHR(x, 3))

/* σ1 (소문자 시그마 1): 메시지 스케줄용 */
#define sigma1(x)    (ROTR(x, 17) ^ ROTR(x, 19) ^ SHR(x, 10))
    
```



SHA256 Quantum circuit

- Lee et. al [1]
 - 다양한 최적화 변형 양자회로 제안(큐비트, T-depth, balanced)
 - Pruning Procedure을 이용하여 adder들의 T-depth 최적화 → SHA256에 사용
- Jang et. al [2]
 - Depth 최적화 양자회로 제안
 - Save adder을 사용한 다중 곱셈단계 최적화
- (공통) Final 단계에서 initial hash value와 라운드 결과 (a ~ h)를 더하는 과정 생략
 - Grover's algorithm 수행에 생략 가능하다고 판단
 - 해시값 Y를 알고있는 상태에서 원본 메시지 X를 찾는 pre-image attack (즉, $H(X)=Y$ 를 만족하는 X찾기)
 - $Y = H + \text{RoundOutput}(X)$ 인 classic data 이므로, $Y - H = \text{RoundOutput}(X)$ 으로 pre-computation 가능

[1] J. Lee, S. Lee, Y.-S. Lee, and D. Choi, "T-depth reduction method for efficient SHA-256 quantum circuit construction," IET Information Security, vol. 17, no. 1, pp. 46–65, 2023.

[2] Jang, Kyungbae, et al. "Quantum implementation and analysis of SHA-2 and SHA-3." IEEE Transactions on Emerging Topics in Computing (2025).

SHA256 Quantum circuit (Lee et. al)

- Pruning Procedure

- 양자회로 최적화 기법 중 하나로, 회로 내 게이트들을 줄이거나 합치는 방식
- Subcircuit 을 식별하여 회로 최적화

[Subcircuit 식별]

1. 시작점 설정 (Start Point)

- 하나의 CNOT게이트 지정 → “Start Point”
- Start Point 의 각 두 개의 Wire을 기준으로 탐색

2. 탐색 방향 및 범위 (Traversing)

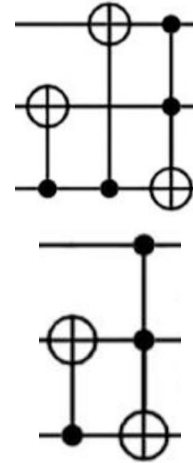
- Start Point (CNOT gate)를 기준으로 좌우로 회로 탐색 시작
- 탐색을 진행하며 Subcircuit에 포함할지 여부 결정, 탐색이 끝나는 지점이 Termination border가 됨

3. 게이트별 처리 규칙 (Rule)

- X , R_z 게이트: 만나는 즉시 통과 (부분회로에 포함)
- H 게이트: 종료지점(Termination point)이 되며, 탐색중단
- CNOT게이트: 1) 공유: Start point(CNOT)와 control, target 라인을 모두 공유 or 종료 경계 안에 포함된 라인이 있을때 → 포함
2) New 연결: 탐색중인 부분과 control 만 공유 → 포함 (타겟 부분이 있는 새로운 라인이 Anchor point가 되어 탐색 라인의 범위가 확장됨)
3) 연결불가: 탐색중인 라인에 target 부분만 있고 control부분이 경계 밖에 있을때 → Skip

SHA256 Quantum circuit (Lee et. al)

- Maj [1] (in-place)
 - $\text{Maj}(x, y, z) = ((x \& y) \wedge ((x \& z) \wedge ((y \& z)))$
 - Toffoli 1개 CNOT 2개
- Ch (in-place)
 - $\text{Ch}(x, y, z) = ((x \& y) \wedge ((\sim(x) \& z)))$
 - CNOT 1개, Toffoli 1개
- Σ_0 (PLU 분해 방식)
 - $\text{Sigma0}(x) = (\text{ROTR}(x, 2) \wedge \text{ROTR}(x, 13) \wedge \text{ROTR}(x, 22))$
- Σ_1 [2] (PLU 분해 방식)
 - $\text{Sigma1}(x) = (\text{ROTR}(x, 6) \wedge \text{ROTR}(x, 11) \wedge \text{ROTR}(x, 25))$
- σ_0 [2] (PLU 분해 방식)
 - $\text{sigma0}(x) = (\text{ROTR}(x, 7) \wedge \text{ROTR}(x, 18) \wedge \text{SHR}(x, 3))$
- σ_1 [2] (PLU 분해 방식)
 - $\text{sigma1}(x) = (\text{ROTR}(x, 17) \wedge \text{ROTR}(x, 19) \wedge \text{SHR}(x, 10))$



```

/* Ch (Choose): x가 1이면 y 선택, 0이면 z 선택 */
#define Ch(x, y, z)  (((x) & (y)) ^ ((~(x)) & (z)))

/* Maj (Majority): x, y, z 중 다수결 */
#define Maj(x, y, z) (((x) & (y)) ^ ((x) & (z)) ^ ((y) & (z)))

/* Σ0 (대문자 시그마 0): 압축 함수용 */
#define Sigma0(x)    (ROTR(x, 2) ^ ROTR(x, 13) ^ ROTR(x, 22))

/* Σ1 (대문자 시그마 1): 압축 함수용 */
#define Sigma1(x)    (ROTR(x, 6) ^ ROTR(x, 11) ^ ROTR(x, 25))

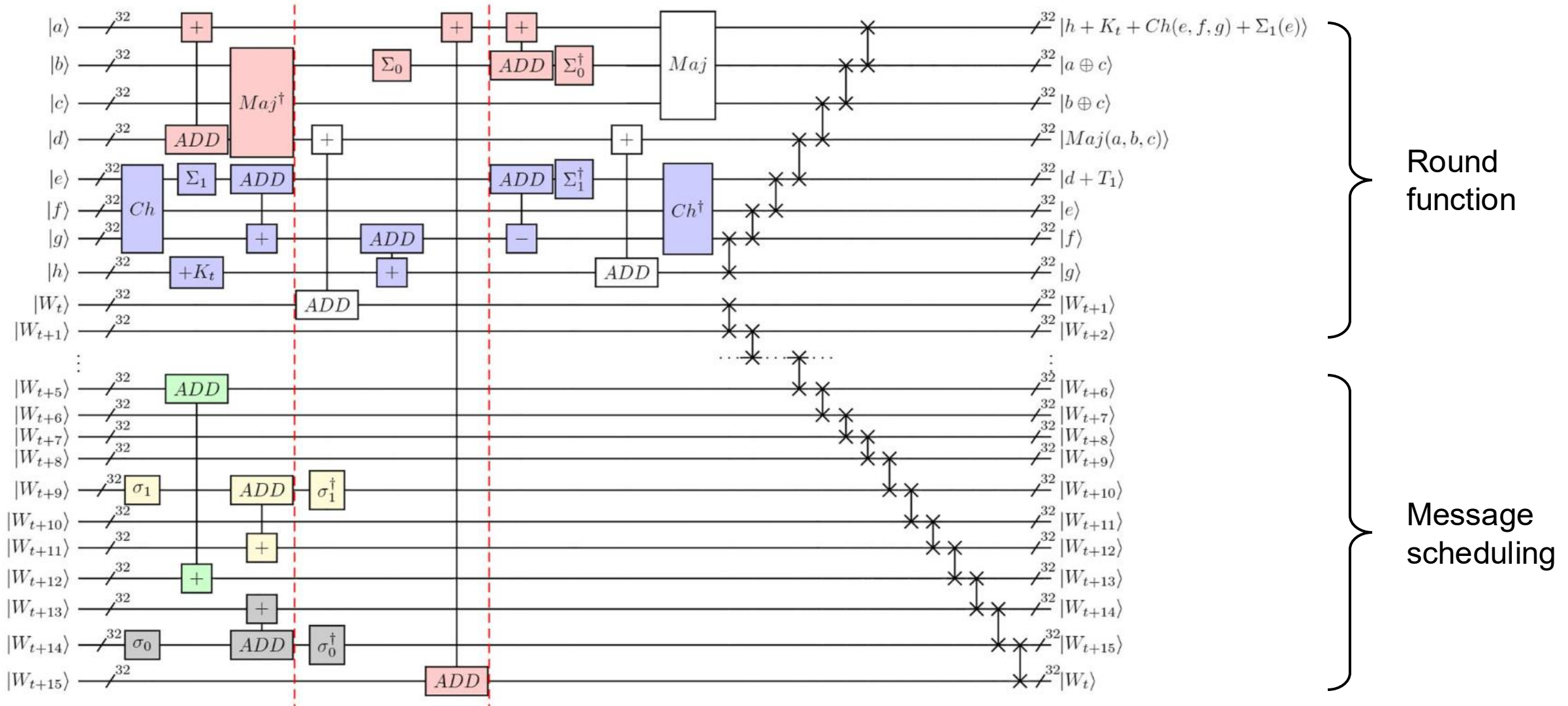
/* σ0 (소문자 시그마 0): 메시지 스케줄용 */
#define sigma0(x)    (ROTR(x, 7) ^ ROTR(x, 18) ^ SHR(x, 3))

/* σ1 (소문자 시그마 1): 메시지 스케줄용 */
#define sigma1(x)    (ROTR(x, 17) ^ ROTR(x, 19) ^ SHR(x, 10))
    
```

TABLE 1 $\Sigma_0, \Sigma_1, \sigma_0, \sigma_1$ function block resources. These function blocks can be implemented using only controlled-NOT gate (CNOT) gates and do not use work qubits at all. These circuits are implemented by arranging CNOT gates in the reverse order of PLU decomposition [4]

	Count	Depth	Swapping
Σ_0	166	55	17
Σ_1	166	44	22
σ_0	193	50	20
σ_1	142	40	23

SHA256 Quantum circuit (Lee et. al)



SHA256 Quantum circuit (Lee et. al)

- 병목 현상이 많은 부분 ($d+T1$)을 먼저 계산 $\rightarrow$ 다음 라운드 먼저 진출하여 병목현상 줄임

$$T1 = h + \Sigma_1(e) + Ch(e,f,g) + K[i] + W[i]$$

$$T2 = \Sigma_0(a) + Maj(a,b,c)$$

$$h = g$$

$$g = f$$

$$f = e$$

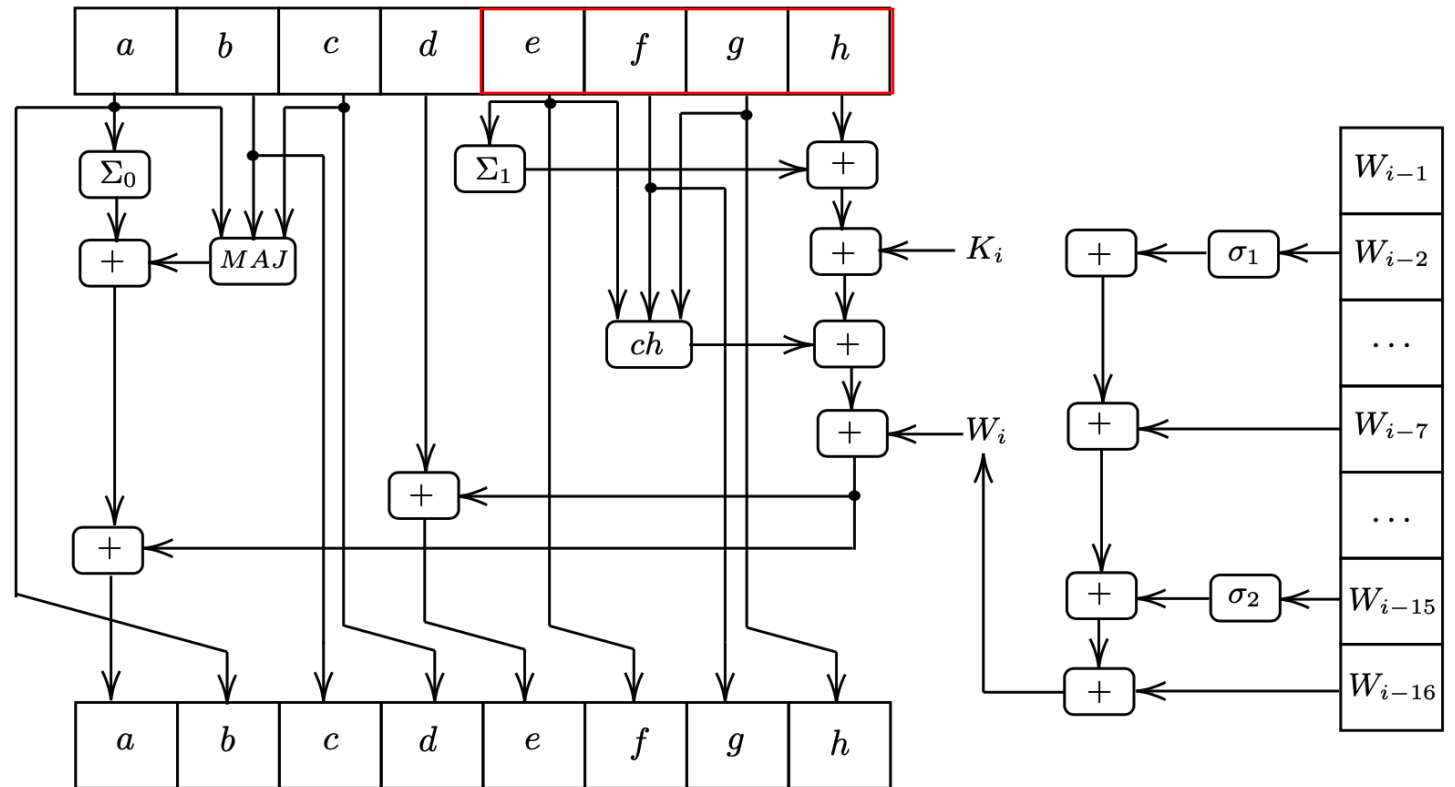
$$e = d + T1$$

$$d = c$$

$$c = b$$

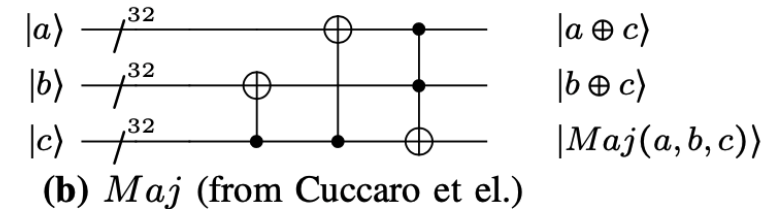
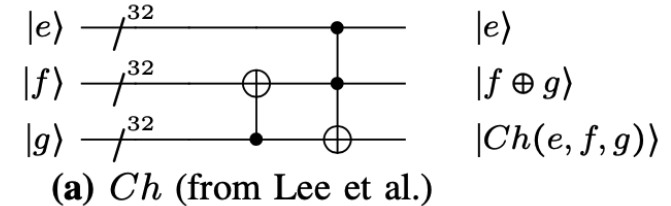
$$b = a$$

$$a = T1 + T2$$



SHA256 Quantum circuit (Jang et. al)

- Maj (in-place, Lee et. al)
 - $\text{Maj}(x, y, z) = (((x) \& (y)) \wedge ((x) \& (z)) \wedge ((y) \& (z)))$
 - Toffoli 1개 CNOT 2개
- Ch (in-place, Cuccaro et. al)
 - $\text{Ch}(x, y, z) = (((x) \& (y)) \wedge ((\sim(x)) \& (z)))$
 - CNOT 1개, Toffoli 1개



- Σ_0 (PLU 분해 방식, **out-of-place**)
 - $\text{Sigma0}(x) = (\text{ROTR}(x, 2) \wedge \text{ROTR}(x, 13) \wedge \text{ROTR}(x, 22))$
- Σ_1 [2] (PLU 분해 방식, **out-of-place**)
 - $\text{Sigma1}(x) = (\text{ROTR}(x, 6) \wedge \text{ROTR}(x, 11) \wedge \text{ROTR}(x, 25))$
- σ_0 [2] (PLU 분해 방식, **out-of-place**)
 - $\text{sigma0}(x) = (\text{ROTR}(x, 7) \wedge \text{ROTR}(x, 18) \wedge \text{SHR}(x, 3))$
- σ_1 [2] (PLU 분해 방식, **out-of-place**)
 - $\text{sigma1}(x) = (\text{ROTR}(x, 17) \wedge \text{ROTR}(x, 19) \wedge \text{SHR}(x, 10))$

Linear layer	Source	#CNOT	#Qubit (reuse)	Depth
Σ_0	Lee et al. (PLU) [5]	166	32	55
	Ours (Out-of-place)	96	64 (32)	3
Σ_1	Lee et al. (PLU) [5]	166	32	44
	Ours (Out-of-place)	96	64 (32)	3
σ_0	Lee et al. (PLU) [5]	193	32	50
	Ours (Out-of-place)	93	64 (32)	3
σ_1	Lee et al. (PLU) [5]	142	32	40
	Ours (Out-of-place)	86	64 (32)	3

SHA256 Quantum circuit (Jang et. al)

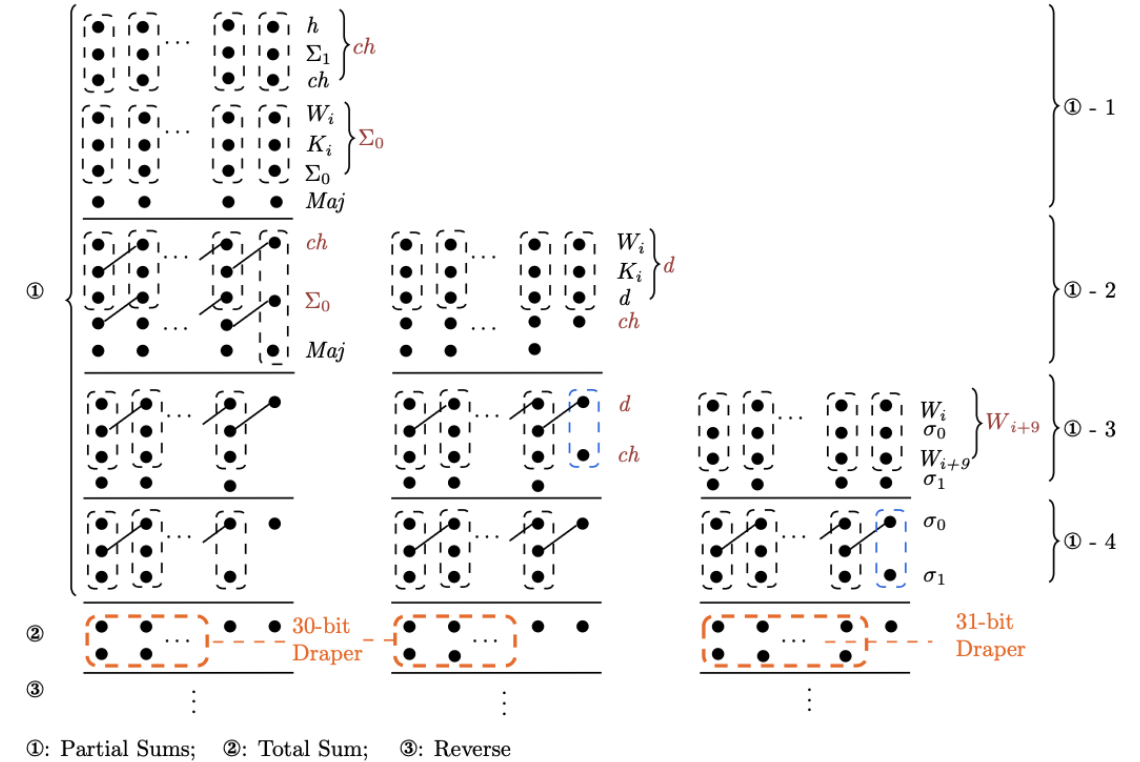
- Critical path

$$h' = \Sigma_0(a) + Maj(a, b, c) + \Sigma_1(e) + Ch(e, f, g) + h + K_i + W_i.$$

$$d' = d + \Sigma_1(e) + Ch(e, f, g) + h + K_i + W_i.$$

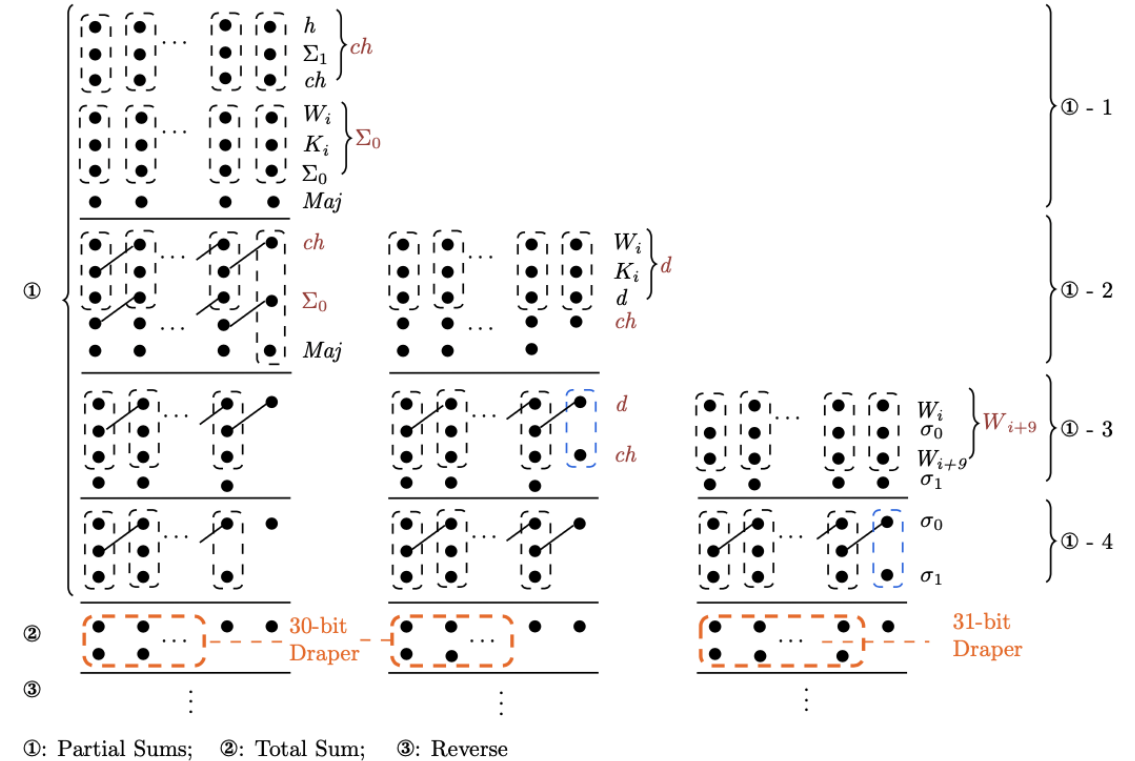
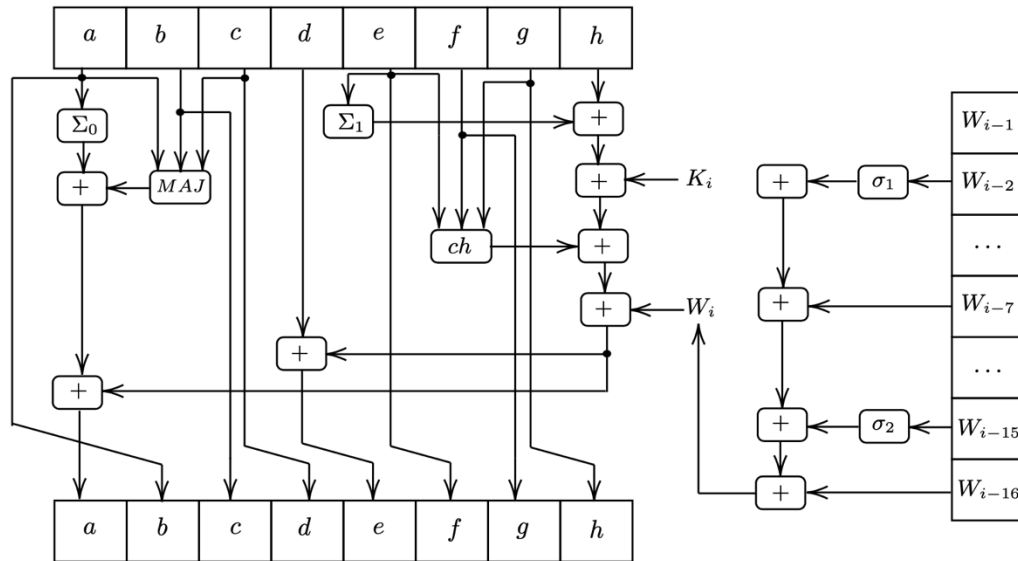
$$W_t = M_t \quad (0 \leq t \leq 15).$$

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16} \quad (16 \leq t \leq 63).$$



SHA256 Quantum circuit (Jang et. al)

$$\begin{aligned}
 h' &= \Sigma_0(a) + Maj(a, b, c) + \Sigma_1(e) + Ch(e, f, g) + h + K_i + W_i \\
 d' &= d + \Sigma_1(e) + Ch(e, f, g) + h + K_i + W_i \\
 W_t &= M_t \quad (0 \leq t \leq 15). \\
 W_t &= \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16} \quad (16 \leq t \leq 63).
 \end{aligned}
 \left. \begin{array}{l} \text{Quantum carry-save adder (QCSA)} \\ \text{Quantum carry-save adder (QCSA)} \\ \text{Quantum carry-save adder (QCSA)} \end{array} \right\} \text{Parallel}$$



Quantum carry-save adder (QCSA)

- QCSA

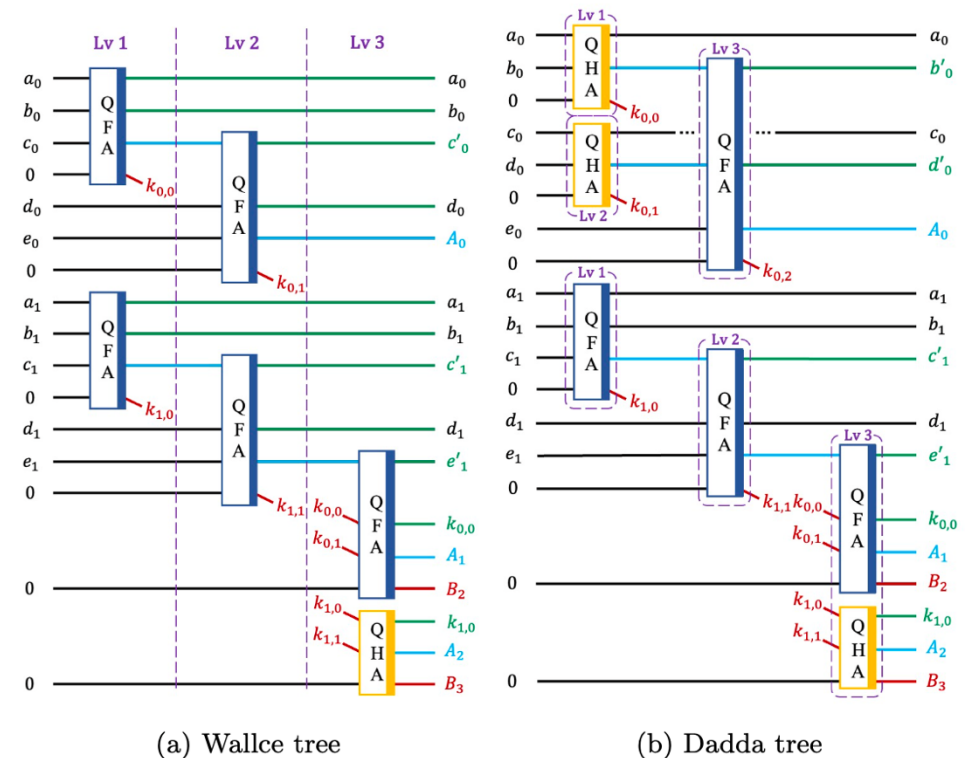
- 각 단계에서 발생하는 Carry를 즉시 전파시키지 않고 다음 단계의 입력으로 넘김 (병렬성)
- (Reduction) QHA와 QFA를 트리 형태로 배치하여, 피연산자의 개수가 2개가 남을 때까지 반복적으로 줄임
- 다수의 피연산자 $\rightarrow$ QCSA(Tree-reduction) $\rightarrow$ 2개의 피연산자 (ADD) $\rightarrow$ 최종결과

- QHA (Quantum Half Adder)

- 2개의 입력을 더해서 합(Sum)과 올림(Carry)를 출력

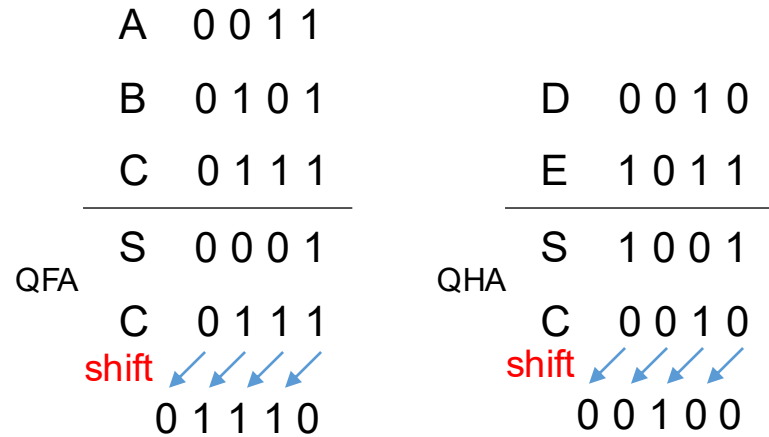
- QFA (Quantum Full Adder)

- 3개의 입력을 더해서 합(Sum)과 올림(Carry)를 출력

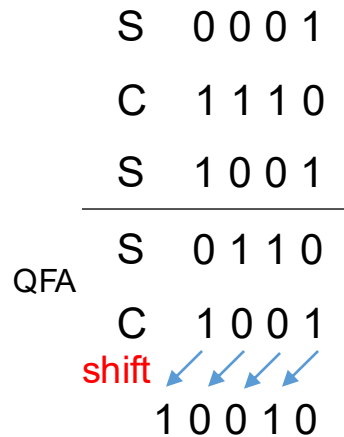


Quantum carry-save adder (QCSA)

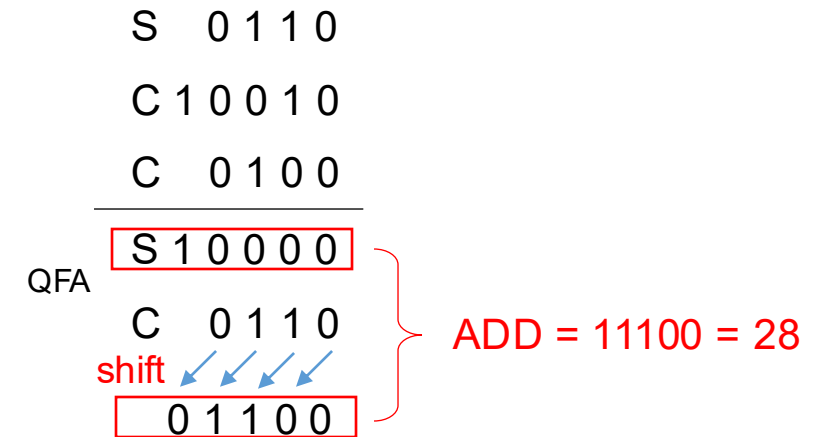
1. $A = 3$ (0011), $B = 5$ (0101), $C = 7$ (0111), $D = 2$ (0010), $E = 11$ (1011)



2. $S = 0001$, $C = 1110$, $S = 1001$, $C = 0100$



3. $S = 0110$, $C = 10010$, $C = 0100$



SHA256 Quantum circuit (Jang et. al)

- 큐비트를 아끼기 위해 특정 라운드 (9~14)에서 $csa1 \rightarrow csa1_0$ 을 사용 (큐비트 감소, depth 증가)
- $csa1$: 덧셈 입력 항목 7개 (W포함), 3개
 - $W[i]$ 를 포함한 7개의 입력의 첫단계를 병렬로 먼저 처리
- $csa1_0$: 덧셈 입력 항목 6개, 4개(W포함)
 - $W[i]$ 를 제외한 6개만 먼저 계산(ancilla 부족, 일부 ancilla 가져와서 사용)

c) *Conditional borrowing technique*: Even after generating all the W_i in the message scheduling, the round function continues to proceed. Thus, we search idle qubits from the round function (Expressions (6)) and then borrow it for the message scheduling (Expressions (7)). We conditionally borrow the idle output qubits from the last round in reverse order, and return them before the round function begins, resulting in a total reduction of 896 ($= 32 \times 28$) qubits. Algorithm 1 formally describes the conditional borrowing Technique.

Algorithm 1: Conditional borrowing in SHA-2-256.

Input: Output qubit set $output_{round}$ in round function

Output: The output qubit set $output_{round}$ borrowed during message scheduling

```
1: Check for idle output qubits in  $output_{round}$ 
2: for the round  $i$  from 0 to 39
3:   if  $i < 28$ 
4:     if  $i < 19$ 
5:       Put  $output_{round}^{(63-i)}$  into  $ancilla_{borrow}^{[2i:2i+1]}$ 
6:     end if
7:     Use  $ancilla_{borrow}^i$  for the  $i$ -th message scheduling
8:   else Allocate  $output_{new}^i$  for the  $i$ -th message scheduling
9:   end if
10: end for
11: for each  $i \in ancilla_{borrow}^{[0:27]}$  in reverse order
12:   Reverse the  $i^{th}$  message scheduling
13: end for
14: return  $ancilla_{borrow}$ 
```

```
elif i >= 1 and i <= 31: # 1~31라운드
    if i >= 1 and i <= 5: # 1~5라운드
        input3 = [W[idx - 16], ancilla_0, ancilla_1]
    elif i == 6 or i == 7: # 6, 7라운드
        input3 = [W[idx - 16], ancilla_0, W[idx - 7], ancilla_1]
    elif i == 8: # 8라운드
        input3 = [W[idx - 16], W[idx - 7], ancilla_1]
    elif i >= 9 and i <= 13: # 9~13라운드
        input3 = [W[idx - 7], ancilla_1]
    elif i == 14: # 14라운드
        input3 = [ancilla_0, W[idx - 7], ancilla_1]
    else: # 15~31라운드
        input3 = [W[idx - 16], ancilla_0, W[idx - 7], ancilla_1]

    if i <= 27: # 1~27라운드
        output = [0, 0, garbages[i]]
    else: # 28~31라운드
        output = [0, 0, 0]

    if i >= 9 and i <= 14: # 9~14라운드
        h[7], h[3], ww = csa1_0(eng, input, input2, input3, add_ancilla, output)
    else: # 15~31라운드
        h[7], h[3], ww = csa1(eng, input, input2, input3, add_ancilla, output)
    W.append(ww)
```

Q & A